

Цель работы: изучить принципы автоматической компоновки элементов на форме.

Теоретические сведения

Классы компоновки виджетов (Layouts) являются одной из сильных сторон Qt. По сути, это контейнеры, которые после изменения размеров окна автоматически приводят в соответствие размеры и координаты виджетов, находящихся в нем. Хотя они ничего не добавляют к функциональной части самой программы, тем не менее, они очень важны для внешнего вида окон приложения. Компоновка определяет расположение различных виджетов относительно друг друга.

Конечно, можно вручную размещать виджеты в окнах приложения, но это существенно усложняет разработку. Ведь тогда, чтобы заново упорядочить элементы, нужно будет отлавливать и обрабатывать изменение размеров окна приложения. Подобные ситуации хорошо известны программистам на языке Visual Basic, которые вынуждены писать для этого сложные методы.

Еще один из недостатков размещения вручную состоит в том, что если приложение поддерживает несколько языков, то, поскольку слова в разных языках имеют разную длину, необходим механизм, который мог бы в процессе работы программы динамически поправлять и изменять размеры и координаты виджетов, — иначе части текста на другом языке могут оказаться «отрезанными». Классы компоновки библиотеки Qt выполняют эту непростую работу за вас. Более того, классы компоновки могут инвертировать направление размещения элементов, что может быть полезно для пишущих справа налево, — например, в Израиле или в странах арабского Востока. Иерархия классов компоновки приведена на рисунке 10.1.

От класса QLayout унаследованы классы QGridLayout и QBoxLayout. Класс QGridLayout управляет табличным размещением, а от QBoxLayout унаследованы два класса QHBoxLayout и QVBoxLayout для горизонтального и вертикального размещения.

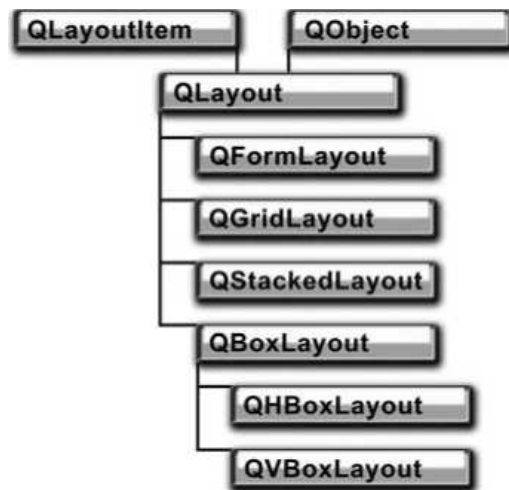


Рис. 10.1. Иерархия классов менеджеров компоновки

Класс QBoxLayout

Объект класса QBoxLayout может управлять как горизонтальным, так и вертикальным размещением. Для того чтобы задать способ размещения, первым параметром конструктора должно быть одно из следующих значений:

- LeftToRight — горизонтальное размещение, заполнение осуществляется слева направо;
- RightToLeft — горизонтальное размещение, заполнение выполняется справа налево;
- TopToBottom — вертикальное размещение, заполнение осуществляется сверху вниз;
- BottomToTop — вертикальное размещение, заполнение выполняется снизу вверх.

Этот класс расширяет класс QLayout методами вставки на заданную позицию:

- Виджета — insertWidget();
- встроенной компоновки — insertLayout();
- расстояния между виджетами — insertspacing();
- фактора растяжения — insertStretch().

К компоновке при помощи метода addSpacing() можно добавить заданное расстояние между двумя виджетами.

Класс QBoxLayout определяет свой собственный метод addwidget() для добавления виджетов в компоновку с возможностью указания, в

дополнительном параметре, фактора растяжения (по умолчанию этот параметр равен нулю).

Горизонтальное размещение QHBoxLayout. Объекты класса QHBoxLayout упорядочивают все виджеты только в горизонтальном порядке — слева направо. Его применение аналогично использованию класса QVBoxLayout, но передавать в конструктор дополнительный параметр, задающий горизонтальный порядок размещения, не нужно. Пример горизонтальной компоновки виджетов приведен на рисунке 10.2.

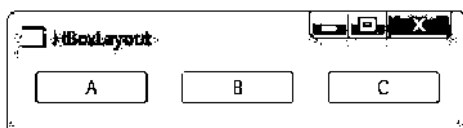


Рис.10.2. Размещение кнопок по горизонтали

Вертикальное размещение QVBoxLayout

Компоновка QVBoxLayout унаследована от QVBoxLayout и упорядочивает все виджеты только по вертикали — сверху вниз. В остальном она ничем не отличается от классов QVBoxLayout и QHBoxLayout. Пример вертикальной компоновки приведен на рисунке 10.3.

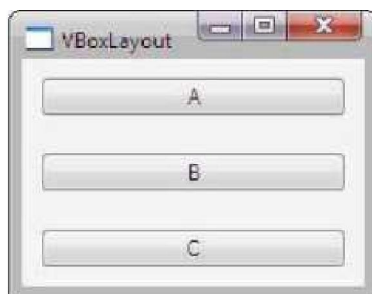


Рис.10.3. Размещение кнопок по вертикали

Факторы растяжения можно самостоятельно добавлять в компоновки, для чего существует метод `addstretcho`. В этом случае фактор растяжения образно можно сравнить с пружиной, которая находится между виджетами и может иметь различную упругость в соответствии с задаваемым параметром. На Рисунке 4 показан пример добавления фактора растяжения между двумя виджетами для расположения виджетов по краям окна.



Рис.10.4. Фактор растяжения

Вложенные размещения

Размещая одну компоновку внутри другой, можно создавать размещения практически любой сложности. Для организации вложенных размещений существует метод `addLayout`, в который вторым параметром передается фактор растяжения для добавляемой компоновки.

На Рисунке 10.5 показан пример вложенного размещения двух менеджеров компоновки. В компоновку `QVBoxLayout` помещается компоновка `QHBoxLayout`. Пример вложенных размещений изображен на рисунке 10.5.

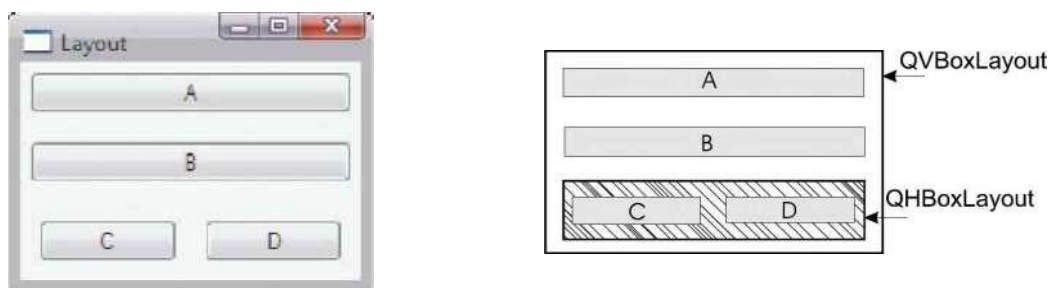


Рис.10.5. Вложенное размещение

Табличное размещение `QGridLayout`

Для табличного размещения используется класс `QGridLayout`, с помощью которого можно быстро создавать сложные по структуре размещения. Таблица состоит из ячеек, позиции которых задаются строками и столбцами.

Добавить виджет в таблицу можно с помощью метода `addWidget()`, передав ему позицию ячейки, в которую помещается виджет. Иногда необходимо, чтобы виджет занимал сразу несколько позиций, чего можно достичь тем же методом `addWidget()`, указав в дополнительных параметрах количество строк и столбцов, которые будет занимать виджет. В последнем параметре можно задать выравнивание (таблица 10.1), например, по центру:
`playout>addWidget(widget, 17, 1, Qt::AlignCenter);`

Таблица 10.1. Значения флагов `Alignment Flag` пространства имен `Qt`

Константа	Значение	Описание
<code>AlignLeft</code>	<code>0x0001</code>	Расположение текста слева
<code>AlignRight</code>	<code>0x0002</code>	Расположение текста справа
<code>AlignHCenter</code>	<code>0x0004</code>	Центровка текста по горизонтали

AlignJustify	0x0008	Растягивание текста по всей
AlignTop	0x0010	Расположение текста вверху
AlignBottom	0x0020	Расположение текста внизу
AlignVCenter	0x0040	Центровка текста по вертикали

Фактор растяжения устанавливается методами `setRowStretch ()` и `setColumnStretch ()`, но не для каждого виджета в отдельности, а для строки или столбца. Расстояние между виджетами также устанавливается для столбцов или строк методом `setspacing ()`.

Пример, показанный на рисунке 10.6, размещает четыре кнопки: А, В, С и D в таблице размером 2 на 2 ячейки.



Рис. 10.6. Размещение кнопок в табличном порядке

Порядок следования табулятора

Пользователь может взаимодействовать с виджетами при помощи мыши и клавиатуры. В последнем случае для выбора нужного виджета используется клавиша табуляции — `<Tab>`, при нажатии которой происходит переход фокуса согласно установленному порядку от одного виджета к другому. Иногда возникает необходимость в изменении этого порядка, который по умолчанию соответствует очередности установки дочерних виджетов в виджете предка. На рис. 10.7 цифрами изображен порядок смены фокуса с помощью табулятора. При появлении диалогового окна с тремя кнопками фокус будет установлен на кнопке С и после нажатия на клавишу табуляции он перейдет на кнопку В, а затем — на кнопку А.



Рис.10.7. а. Порядок смены фокуса, б. Измененный порядок смены фокуса

Изменить порядок смены фокуса можно с помощью статического метода `QWidget::setTabOrder ()`, получающего в качестве параметров два

указателя на виджеты. Следующие вызовы изменяют порядок следования табулятора, на более логичный порядок:

```
QWidget::setTabOrder(A, B);  
QWidget::setTabOrder(B, C);
```

Ход работы

Часть I

Рассмотрим несколько примеров программ, которые демонстрируют основные возможности автоматической компоновки элементов на форме.

Компоновка с помощью `QBoxLayout`.

Данный пример реализует компоновку 6 кнопок на основной форме, в два ряда. Первый ряд располагает элементы слева на право, а второй с права на лево.

Programm10_1.pro

```
QT      += core gui  
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets  
TARGET = Programm10_1  
TEMPLATE = app  
SOURCES += main.cpp\  
          widget.cpp  
HEADERS  += widget.h
```

main.cpp

```
#include "widget.h"  
#include <QApplication>  
  
int main(int argc, char *argv[])  
{  
    QApplication a(argc, argv);  
    Widget w;  
    w.show();  
  
    return a.exec();  
}
```

widget.h

```
#ifndef WIDGET_H
#define WIDGET_H

#include <QWidget>
#include <QBoxLayout>
#include <QPushButton>

class Widget : public QWidget
{
    Q_OBJECT

public:
    Widget(QWidget *parent = 0);
    ~Widget();

private:
    QBoxLayout *mainLayout=NULL;

    QBoxLayout *layout1=NULL;
    QPushButton *buttonA=NULL;
    QPushButton *buttonB=NULL;
    QPushButton *buttonC=NULL;

    QBoxLayout *layout2=NULL;
    QPushButton *buttonD=NULL;
    QPushButton *buttonE=NULL;
    QPushButton *buttonF=NULL;

};

#endif // WIDGET_H
```

widget.cpp

```
#include "widget.h"

Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
```

```

mainLayout = new QVBoxLayout(QBoxLayout::TopToBottom,this);

layout1 = new QVBoxLayout(QBoxLayout::LeftToRight);
mainLayout->addLayout(layout1);

buttonA = new QPushButton(tr("Button A"),this);
layout1->addWidget(buttonA);

buttonB = new QPushButton(tr("Button B"),this);
layout1->addWidget(buttonB);

buttonC = new QPushButton(tr("Button C"),this);
layout1->addWidget(buttonC);

layout2 = new QVBoxLayout(QBoxLayout::RightToLeft);
mainLayout->addLayout(layout2);

buttonD = new QPushButton(tr("Button D"),this);
layout2->addWidget(buttonD);

buttonE = new QPushButton(tr("Button E"),this);
layout2->addWidget(buttonE);

buttonF = new QPushButton(tr("Button F"),this);
layout2->addWidget(buttonF);
}

Widget::~Widget()
{
    delete buttonA;
    delete buttonB;
    delete buttonC;
    delete buttonD;
    delete buttonE;
    delete buttonF;

    delete layout1;
    delete layout2;
}

```

Файлы Programm10_1.pro и main.cpp отдельно рассматривать не стоит, они типичны и хорошо описаны в предыдущих лабораторных.

Рассмотри класс основного окна программы. Вначале файла `widget.h` производим подключение необходимых библиотек, для использования следующих классов:

- `QWidget` – класс-родитель для класса основного окна;
- `QBoxLayout` класс автоматической компоновки элементов на форме;
- `QPushButton` – класс кнопки.

Затем в структуре класса основного окна объявляются необходимые указатели: 3 указателя на менеджеры компоновки и указатели на 6 кнопок. Указатель `mainLayout` будет содержать объект основного менеджера компоновки, на нем будут находиться оставшиеся классы компоновки. Объект `layout1` будет отвечать за компоновку первой строки виджетов, а `layout2` соответственно за компоновку виджетов второй строки.

Теперь рассмотри конструктор класса основного окна программы. Сначала создается основной объект компоновки:

```
mainLayout = new QBoxLayout(QBoxLayout::TopToBottom,this);
```

Элементы будут на нем располагаться вертикально и порядок заполнения будет сверху в низ (за это отвечает опция `QBoxLayout::TopToBottom`, переданная в конструктор). Нам не следует явно задавать что данный объект будет основным для виджета окна, т.к. в параметрах указателя основное окно программы передано в качестве родителя.

Затем создается объект компоновки первой строки виджетов и укладывается на основной менеджер компоновки:

```
layout1 = new QBoxLayout(QBoxLayout::LeftToRight);  
mainLayout->addLayout(layout1);
```

Виджеты, которые будут уложены на него, будут располагаться горизонтально, с порядком заполнения с лева на право.

Аналогичным образом поступаем и со вторым объектом компоновки:

```
Layout2 = new QBoxLayout(QBoxLayout::RightToLeft);  
mainLayout->addLayout(layout2);
```

Только, в отличии от первого, порядок заполнения элементами у него установлен с право на лево. Затем создаем кнопки и укладываем их на соответствующие объекты компоновки:

```
buttonA = new QPushButton(tr("Button A"),this);  
layout1->addWidget(buttonA);  
.  
.  
.  
buttonF = new QPushButton(tr("Button F"),this);  
layout2->addWidget(buttonF);
```

В результате выполнения данной программы мы получим форму, на которой виджеты (кнопки) расположены в два ряда. В первом ряду порядок кнопок будет с лева на право, а во втором наоборот.

Часть II

Компоновка с помощью QVBoxLayout и QHBoxLayout.

Рассмотрев более общий класс компоновки виджетов, рассмотрим специализированные QVBoxLayout и QHBoxLayout. Как уже было сказано в теоритической части первый класс компоновки осуществляет вертикальное расположение виджетов, а второй горизонтальное. Пример применения данных классов для размещения виджетов на форме.

Programm10_2.pro

```
QT      += core gui
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
TARGET = Programm10_2
TEMPLATE = app
SOURCES += main.cpp\
            widget.cpp
HEADERS  += widget.h
```

main.cpp

```
#include "widget.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    Widget w;
    w.show();

    return a.exec();
}
```

widget.h

```
#ifndef WIDGET_H
#define WIDGET_H
```

```

#include <QWidget>
#include <QHBoxLayout>
#include <QVBoxLayout>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>

class Widget : public QWidget
{
    Q_OBJECT

public:
    Widget(QWidget *parent = 0);
    ~Widget();

private:
    QVBoxLayout *mainLayout=NULL;
    QHBoxLayout *layout1=NULL;
    QLabel *labelA=NULL;
    QLineEdit *editA=NULL;
    QHBoxLayout *layout2=NULL;
    QPushButton *buttonSetA=NULL;
};

#endif // WIDGET_H

```

widget.cpp

```

#include "widget.h"

Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    mainLayout = new QVBoxLayout(this);

    layout1 = new QHBoxLayout;
    mainLayout->addLayout(layout1);

    labelA = new QLabel(tr("Enter A"),this);
    layout1->addWidget(labelA);
    editA = new QLineEdit(this);
    layout1->addWidget(editA);
}

```

```

layout2 = new QHBoxLayout;
mainLayout->addLayout(layout2);

layout2->addStretch();
buttonSetA = new QPushButton(tr("Set A"),this);
layout2->addWidget(buttonSetA);
}

Widget::~Widget()
{

}

```

Как обычно, в заголовочном файле класс основного окна произведем подключение необходимых библиотек для данного проекта:

- QWidget – класс-родитель класса основного окна;
- QHBoxLayout – горизонтальный компоновщик виджетов;
- QVBoxLayout – вертикальный компоновщик виджетов;
- QLabel – класс текстовой метки;
- QLineEdit – класс текстового поля;
- QPushButton – класс кнопки.

В структуре класса основного окна произведем объявление указателей на необходимые объекты.

В конструкторе класса основного окна произведем создание объектов и расположение их на форме.

Сначала создаем основной компоновщик виджетов:

```
mainLayout = new QVBoxLayout(this);
```

Затем создаем компоновщики первой и второй строки и располагаем их на основном компоновщике:

```

layout1 = new QHBoxLayout;
mainLayout->addLayout(layout1);
layout2 = new QHBoxLayout;
mainLayout->addLayout(layout2);

```

Создаем элементы (виджеты) и располагаем их на нужных компоновщиках. В первой строке расположим текстовую метку и текстовое поле. Их создание типичное, и не требует дополнительных пояснений. Во второй строке расположим кнопку. Однако необходимо сделать так, чтобы при изменении ширины формы кнопка не меняла своего размера.

```

layout2->addStretch();
buttonSetA = new QPushButton(tr("Set A"),this);

```

```
layout2->addWidget(buttonSetA);
```

Для этого перед расположением формы создадим фактор растяжения (`layout2->addStretch()`), а только после этого создадим объект кнопки и расположим его на компоновщике.

Часть III

Компоновка с помощью `QGridLayout`.

Данный метод компоновки элементов на форме позволяет выравнивать положение виджетов по сетке. Рассмотрим пример.

Programm10_3.pro

```
QT      += core gui
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
TARGET = Programm10_3
TEMPLATE = app
SOURCES += main.cpp\
           widget.cpp
HEADERS  += widget.h
```

main.cpp

```
#include "widget.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    Widget w;
    w.show();

    return a.exec();
}
```

widget.h

```
#ifndef WIDGET_H
#define WIDGET_H

#include <QWidget>
```

```

#include <QGridLayout>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>

class Widget : public QWidget
{
    Q_OBJECT

public:
    Widget(QWidget *parent = 0);
    ~Widget();

private:
    QGridLayout *mainLayout=NULL;
    QLabel *labelA=NULL;
    QLineEdit *editA=NULL;
    QLabel *labelB=NULL;
    QLineEdit *editB=NULL;
    QPushButton *buttonSet=NULL;
};

#endif // WIDGET_H

```

widget.cpp

```

#include "widget.h"

Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    mainLayout = new QGridLayout(this);

    labelA = new QLabel(tr("Enter A"),this);
    mainLayout->addWidget(labelA,0,0,1,1);
    editA = new QLineEdit(this);
    mainLayout->addWidget(editA,0,1,1,2);

    labelB = new QLabel(tr("Enter B"),this);
    mainLayout->addWidget(labelB,1,0,1,1);
    editB = new QLineEdit(this);
    mainLayout->addWidget(editB,1,1,1,2);
}

```

```

        buttonSet = new QPushButton(tr("Set"),this);
        mainLayout->addWidget(buttonSet,2,2,1,1);
    }

    Widget::~Widget()
    {

    }

```

Принцип использования QGridLayout аналогичен предыдущим, однако при расположении виджета необходимо указывать дополнительные параметры.

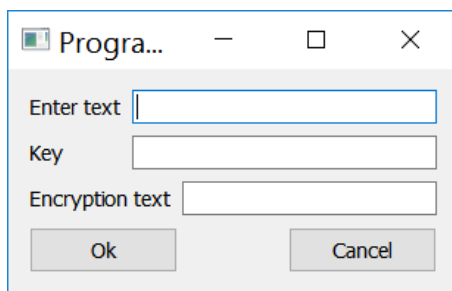
```
mainLayout->addWidget(labelA,0,0,1,1);
```

Первый параметр, как и у всех компоновщиков, является указателем на виджет. Затем обязательно указывается строка и столбец, в котором необходимо расположить элемент, а также количество ячеек, которые должен занимать виджет по горизонтали и вертикали. В данном случае виджет labelA располагается в 0 строке, 0 столбце и занимает по одной ячейке в каждом направлении.

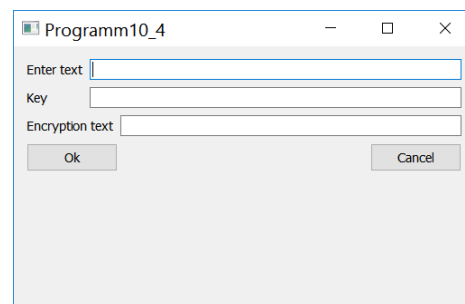
Часть IV

Задание для самостоятельной работы

Реализовать программу, которая будет создавать форму, аналогичную той, что изображена на рисунке 10.8.



А



Б

Рис.10.8. Вид формы окна для задания на самостоятельную работу. А) Исходное состояние формы. Б) Вид формы при изменении размера

Содержание отчета

1. Наименование и цель работы.
2. Краткая основная теоретическая часть, которая применялась вами при выполнении работы.
3. Листинги исходного кода файлов проектов.
4. Скриншоты окон приложений для каждого пункта лабораторной работы.
5. Вывод о полученных навыках и знаниях после выполнения работы.

Контрольные вопросы для самопроверки

1. Что такое классы управления автоматического размещения элементов?
2. Особенности применения QVBoxLayout?
3. Особенности применения QHBoxLayout?
4. Особенности применения QVBoxLayout?
5. Особенности применения QGridLayout?
6. Что такое фактор растяжения?
7. Как задается порядок следования табулятора?